第8回: セキュリティの基礎 & 総仕上げ

Webアプリケーション基礎 2026

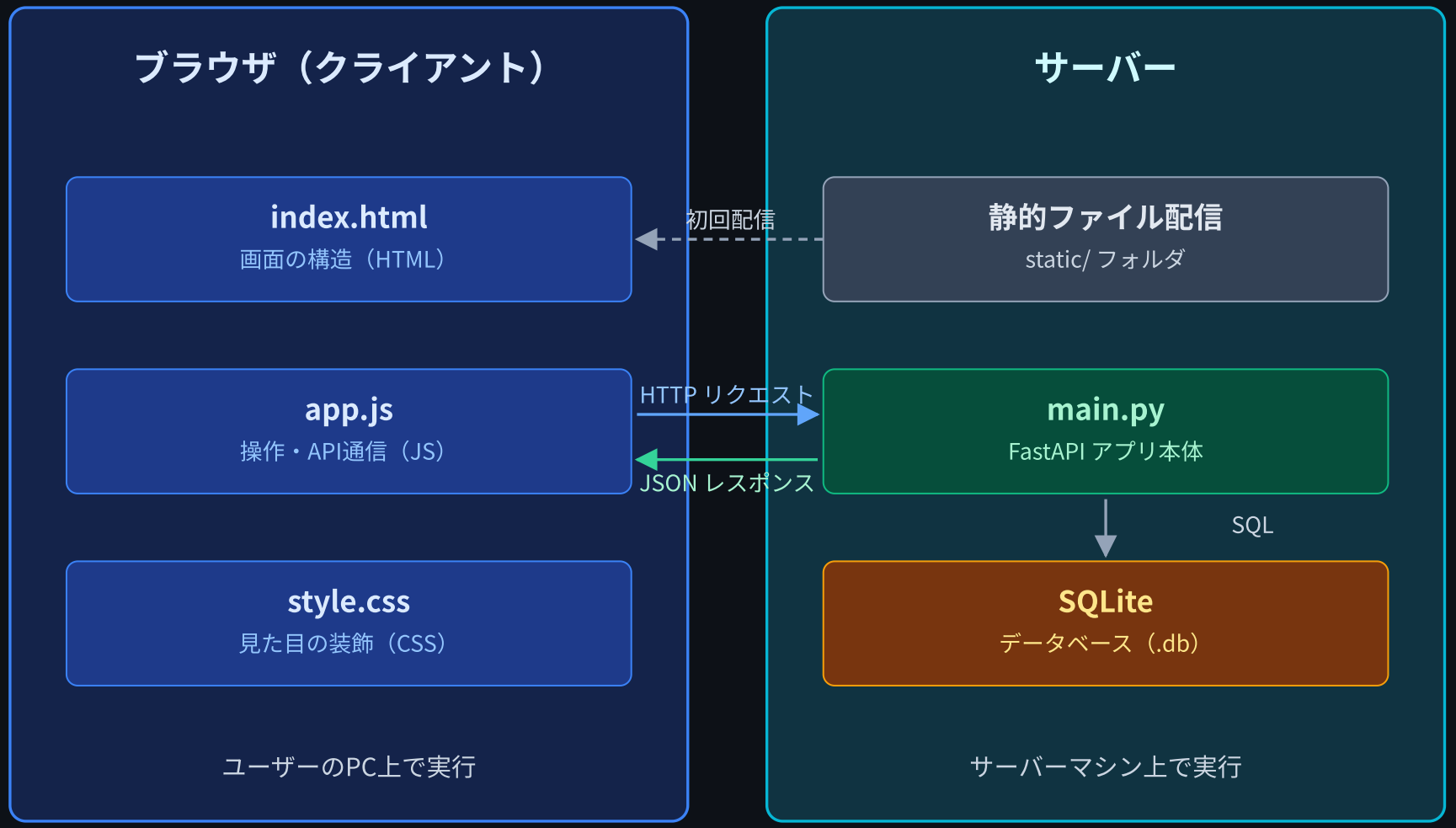
今日のゴール

セキュリティ対策を施し、TODOアプリを**完全体**にする

TODOの追加・完了・削除ができる！ だが...

- 悪意のある入力への対策がない
- エラー時の処理が甘い
- バリデーションが不十分

> 今日はこの「守りの部分」を強化して、アプリを完成させます。


ブラウザ（クライアント）
index.html
画面の構造（HTML）
app.js
操作・API通信（JS）
style.css
見た目の装飾（CSS）
ユーザーのPC上で実行
初回配信
HTTP リクエスト
JSON レスポンス
サーバー
静的ファイル配信
static/ フォルダ
main.py
FastAPI アプリ本体
SQL
SQLite
データベース（.db）
サーバーマシン上で実行

今日の流れ

前半

- Webセキュリティ概論
- XSS（クロスサイトスクリプティング）
- SQLインジェクション

後半

- バリデーション & HTTPS
- エラーハンドリング
- 総まとめ & 完成デモ

> **キーワード:** 「入力は信用するな」

前半: Webセキュリティの基礎

1. Webセキュリティ概論

なぜセキュリティが重要なのか？

セキュリティ事故の実例

- **個人情報の漏洩**: データベースから何万件ものユーザー情報が盗まれる
- **Webサイトの改ざん**: 正規サイトに悪意のあるスクリプトが仕込まれる
- **サービス停止**: 攻撃によりサービスが利用不能になる

他人事ではない

- 小さなWebアプリでも攻撃の対象になり得る
- **セキュリティは「あとから」ではなく「最初から」考えるべき**
- 「動く」だけでは不十分。「安全に動く」ことが重要

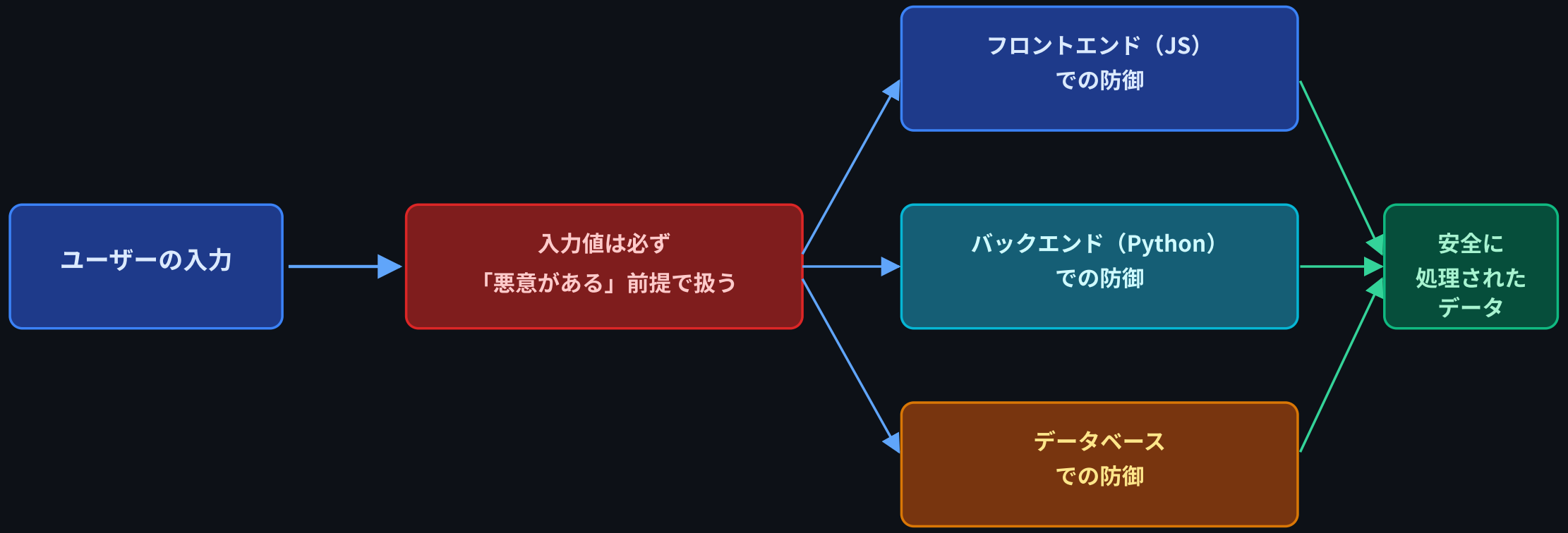
代表的なWeb攻撃

攻撃手法	攻撃場所	概要
XSS（クロスサイトスクリプティング）	クライアント側	悪意のあるスクリプトをWebページに注入する
SQLインジェクション	サーバー側	不正なSQL文をデータベースに送り込む
CSRF（クロスサイトリクエストフォージェリ）	両方	ユーザーになりすまして不正リクエストを送る

今日は特に **XSS** と **SQLインジェクション** を詳しく学びます。

セキュリティの大原則

「入力は信用するな」（Never Trust User Input）



- テキスト入力欄に「普通のテキスト」が入るとは限らない
- **HTMLタグ、スクリプト、SQL文**が入力される可能性がある

どこで何を守るか

ブラウザ（クライアント）
防御1: XSS対策（textContent使用）
防御2: 入力バリデーション（空文字等）
防御3: エラーハンドリング
HTTP通信
サーバー（Python / FastAPI）
防御4: バリデーション（Pydantic）
防御5: SQLインジェクション対策
防御6: エラーハンドリング（HTTPステータス）
防御7: HTTPS（通信の暗号化）

実習1（10分）: XSS攻撃を体験する

TODOアプリに悪意のある入力をしてみましょう。

手順

1. データベースを初期化してサーバーを起動する

```
cd session08/exercise
python init_db.py   # 第7回からのDBが残っている場合は作り直し
python main.py
```

2. ブラウザでTODOアプリを開く
3. TODO入力欄に以下を入力して「追加」を押す

```
<script>alert(`XSS!`)</script>
```

4. 何が起こったか観察する
5. 次に以下を試す

```
<img src=x onerror="alert(`XSS!`)">
```

2. XSS（クロスサイトスクリプティング）

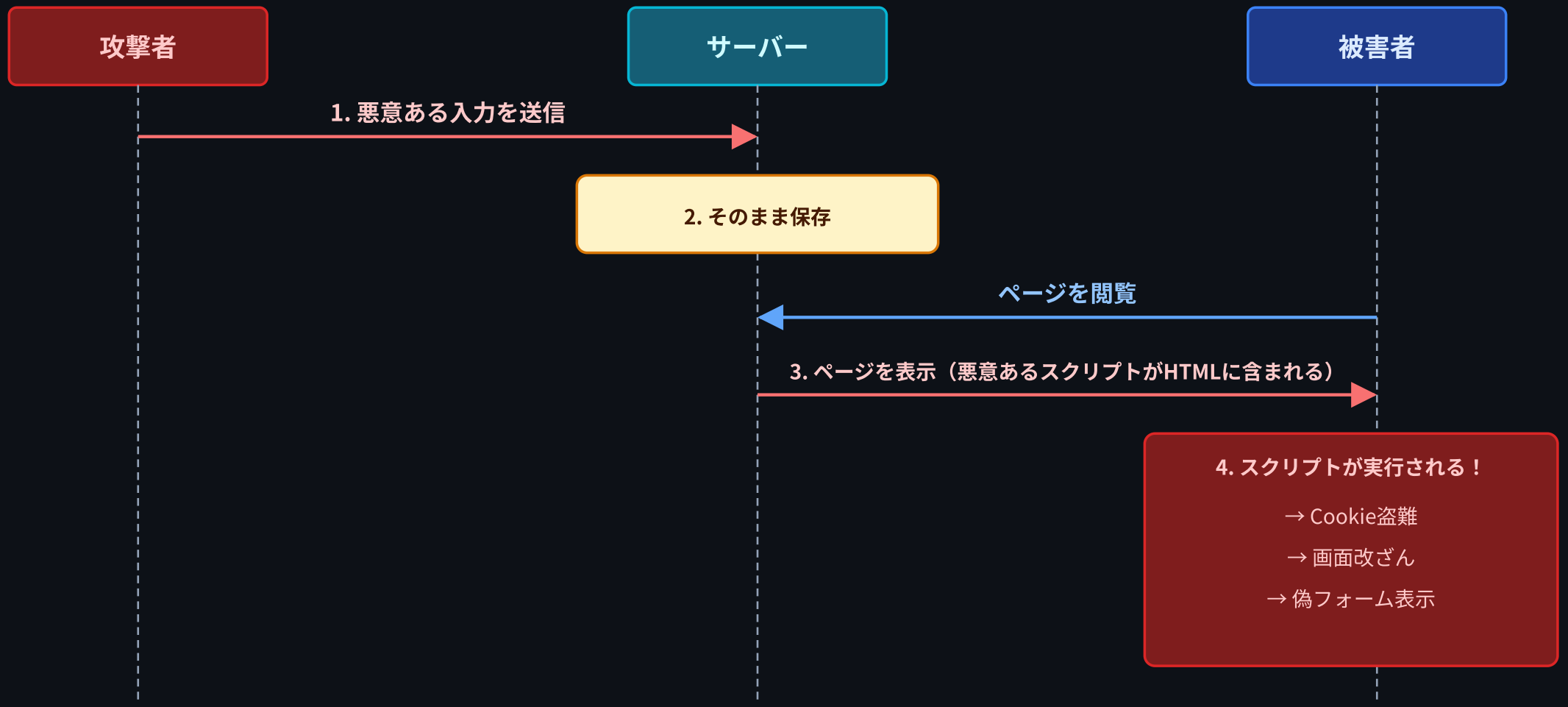
攻撃の仕組みと対策を学ぶ

XSSとは

Cross-Site Scripting（クロスサイトスクリプティング）

ユーザーの入力がそのままHTMLに埋め込まれることを悪用して、
悪意のあるJavaScriptを他のユーザーのブラウザで実行させる攻撃。

XSS攻撃のフロー


攻撃者
サーバー
被害者
1. 悪意ある入力を送信
2. そのまま保存
ページを閲覧
3. ページを表示（悪意あるスクリプトがHTMLに含まれる）
4. スクリプトが実行される！
→ Cookie盗難
→ 画面改ざん
→ 偽フォーム表示

なぜ `innerHTML` は危険か

```
// 危険なコード
const todoText = '<script>alert("XSS!")</script>';
element.innerHTML = todoText;
// → ブラウザがHTMLとして解釈し、スクリプトが実行される可能性がある
```

`innerHTML` は文字列を **HTMLとして解釈** する。
つまり、ユーザーの入力にHTMLタグやスクリプトが含まれていると、
そのまま実行されてしまう。

特に危険な入力パターン

```
<!-- scriptタグ -->
<script>document.cookie を外部に送信</script>

<!-- イベントハンドラ -->
<img src=x onerror="悪意のあるコード">

<!-- リンク偽装 -->
<a href="javascript:悪意のあるコード">クリック</a>
```

XSS対策: `textContent` を使う

```
// 危険: innerHTML（HTMLとして解釈される）
element.innerHTML = userInput;

// 安全: textContent（テキストとして扱われる）
element.textContent = userInput;
```

違いを図解

```
ユーザー入力: "<script>alert('XSS')</script>"

innerHTML の場合:
  → DOM: <script>alert('XSS')</script>  ← HTMLとして解釈！
  → スクリプトが実行される可能性

textContent の場合:
  → DOM: "&lt;script&gt;alert('XSS')&lt;/script&gt;"
  → 画面表示: "<script>alert('XSS')</script>"  ← ただの文字列
```

`textContent` はHTMLタグを自動的にエスケープしてくれる。

コード比較: 脆弱なコード（innerHTML使用）

```
function renderTodos(todos) {
  const list = document.getElementById('todo-list');
  list.innerHTML = '';
  todos.forEach(todo => {
    const li = document.createElement('li');
    // 危険！ ユーザー入力をHTMLとして挿入
    li.innerHTML = `<span>${todo.title}</span>`;
    list.appendChild(li);
  });
}
```

コード比較: 安全なコード（textContent使用）

```
function renderTodos(todos) {
  const list = document.getElementById('todo-list');
  list.innerHTML = '';  // ここはOK: ユーザー入力ではない
  todos.forEach(todo => {
    const li = document.createElement('li');
    const span = document.createElement('span');
    // 安全！ テキストとして挿入
    span.textContent = todo.title;
    li.appendChild(span);
    list.appendChild(li);
  });
}
```

XSS対策のまとめ

方法	説明
`textContent`	ユーザー入力をテキストとして表示する（最も基本）
`createElement`	要素をJavaScriptで組み立てる（innerHTMLの代替）
エスケープ処理	`<` を `<` に変換するなど
CSP	Content-Security-Policy ヘッダー（発展的内容）

ポイント

- ユーザーの入力を `innerHTML` に直接入れない
- DOM要素を組み立てるときは `createElement` + `textContent`
- **サーバー側でもエスケープするとより安全**（多層防御）

実習2（10分）: XSS対策を実装する

手順

1. `exercise/static/app.js` を開く
2. `renderTodos` 関数内の `innerHTML` を見つける
3. `textContent` と `createElement` に書き換える

```
// TODO: この部分を安全な実装に修正してください
// 修正前（危険）:
//   li.innerHTML = `<span>${todo.title}</span>`;
// 修正後（安全）:
const span = document.createElement('span');
span.textContent = todo.title;
li.appendChild(span);
```

4. 再度 `<script>alert('XSS!')</script>` を入力して追加
5. アラートが出ず、**テキストとして表示**されることを確認
6. 開発者ツールのElementsタブで、タグがエスケープされていることを確認

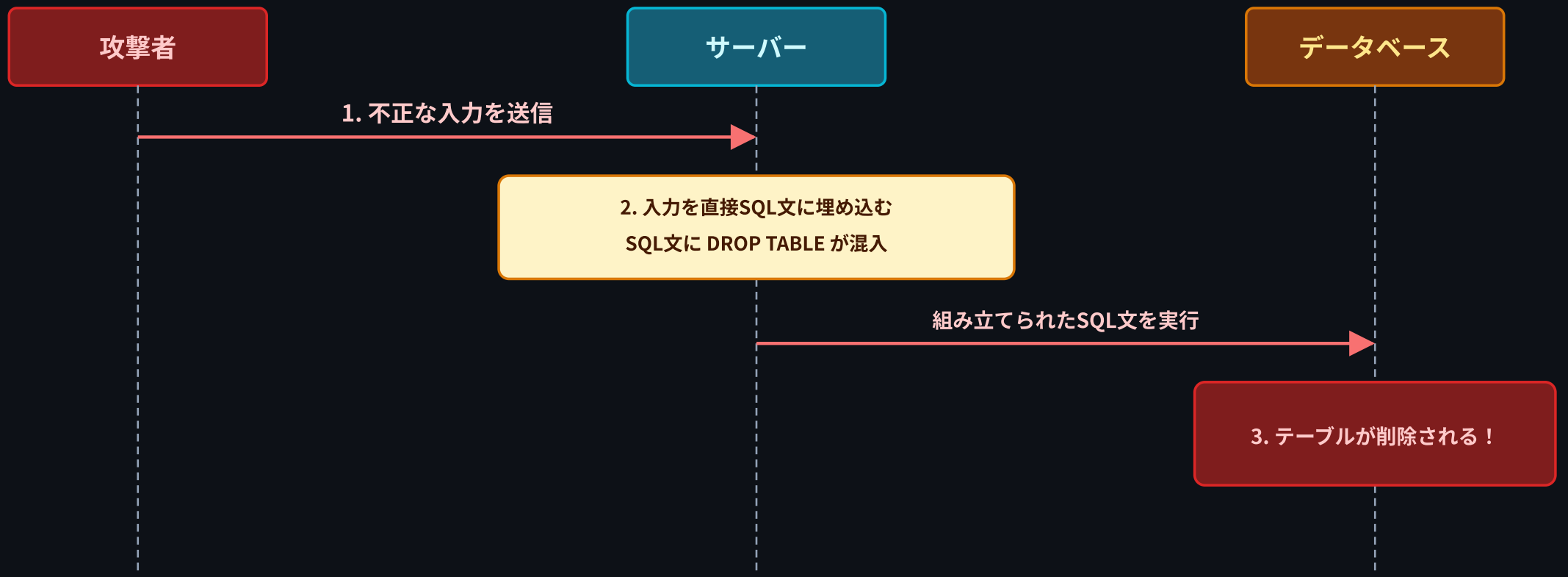
3. SQLインジェクション

サーバー側の攻撃を学ぶ

SQLインジェクションとは

ユーザーの入力が**SQL文の一部として実行**されてしまう攻撃。

攻撃のフロー



危険なコード: 文字列結合でSQLを組み立てる

```python
# 危険！ 絶対にこう書いてはいけない
def add_todo(title: str):
    cursor.execute(
        f"INSERT INTO todos (title, done) VALUES ('{title}', 0)"
    )
```

正常な入力の場合

```
title = "買い物"
→ INSERT INTO todos (title, done) VALUES ('買い物', 0)   ← 期待通り
```

危険なコード: 悪意のある入力の場合

```
title = "'; DROP TABLE todos; --"
→ INSERT INTO todos (title, done) VALUES ('''; DROP TABLE todos; --', 0)
                                          ^^  ^^^^^^^^^^^^^^^^^^^^^^^^^^
                                          空   テーブル削除！
```

`--` はSQLのコメントなので、残りの `', 0)` は無視される。

文字列結合でSQL文を組み立てると、入力値がSQL文の一部として解釈されてしまう。

安全なコード: パラメータバインディング

```
# 安全！ パラメータバインディングを使う
def add_todo(title: str):
    cursor.execute(
        "INSERT INTO todos (title, done) VALUES (?, 0)",
        (title,)
    )
```

パラメータバインディングの仕組み

```
1. SQL文のテンプレート → "INSERT INTO todos (title, done) VALUES (?, 0)"
2. パラメータを別途送信 → ("'; DROP TABLE todos; --",)
3. DBがパラメータを「値」として安全に処理 → SQL文としては実行されない！
```

`?` **プレースホルダ**を使うことで、入力がSQL文の一部として解釈されるのを防ぐ。

コード比較: 脆弱なコード（文字列結合）

```
def get_todos():
    cursor.execute("SELECT * FROM todos WHERE title LIKE '%" + search + "%'")

def add_todo(title):
    cursor.execute(f"INSERT INTO todos (title, done) VALUES ('{title}', 0)")

def delete_todo(id):
    cursor.execute(f"DELETE FROM todos WHERE id = {id}")
```

コード比較: 安全なコード（パラメータバインディング）

```
def get_todos(search):
    cursor.execute("SELECT * FROM todos WHERE title LIKE ?", ('%' + search + '%',))

def add_todo(title):
    cursor.execute("INSERT INTO todos (title, done) VALUES (?, 0)", (title,))

def delete_todo(id):
    cursor.execute("DELETE FROM todos WHERE id = ?", (id,))
```

すべてのSQL操作で `?` プレースホルダを使うことが重要。

SQLインジェクション対策のまとめ

方法	説明
パラメータバインディング	`?` プレースホルダを使う（最も基本・重要）
ORM の使用	SQLAlchemy等を使えば自動的に安全（発展的内容）
入力値バリデーション	許可する文字種・長さを制限する
最小権限の原則	DBユーザーに必要最小限の権限のみ付与

重要なポイント

- **文字列結合でSQL文を組み立てない**（f-string, format, + 連結すべてNG）
- 必ず `?` （SQLite）や `%s` （MySQL/PostgreSQL）を使う
- FastAPIを使う場合、Pydanticでバリデーション → パラメータバインディングの二段構え

実習3（5分）: SQLインジェクションを体験する

手順

1. サーバーが起動していることを確認（`python main.py`）
2. ブラウザでTODOアプリを開く
3. TODO入力欄に以下を入力して「追加」を押す

```
ダミー', 0), ((SELECT sqlite_version()), 0) --
```

4. **何が起こったか観察する**
   - TODOが**2件**追加されていないか？
   - 1件目: 「ダミー」
   - 2件目: SQLiteのバージョン番号（例: `3.40.1`）
5. これは `INSERT` 文が以下のように組み立てられたため:

```
INSERT INTO todos (title, done) VALUES ('ダミー', 0),
  ((SELECT sqlite_version()), 0) --', 0)
```

→ **1つのINSERTで2行が挿入され、DB内部情報が漏洩した！**

実習4（10分）: SQLインジェクション対策を実装する

手順

1. `exercise/main.py` を開く
2. `# TODO(実習4): パラメータバインディングに修正` のコメントを探す
3. **すべての**f-string を `?` プレースホルダに修正する

```
# 修正前（危険）:
cursor.execute(f"INSERT INTO todos (title, done) VALUES ('{todo.title}', 0)")

# 修正後（安全）:
cursor.execute("INSERT INTO todos (title, done) VALUES (?, 0)", (todo.title,))
```

4. `UPDATE` と `DELETE` も同様に修正する
5. サーバーを再起動し、同じ攻撃文字列を入力する
6. 今度は攻撃文字列が**そのままテキストとして**保存されることを確認

後半: バリデーション・エラーハンドリング・総まとめ

4. バリデーション & HTTPS

入力値を正しく検証する

なぜバリデーションが必要か

ユーザーの入力をそのまま信用すると...

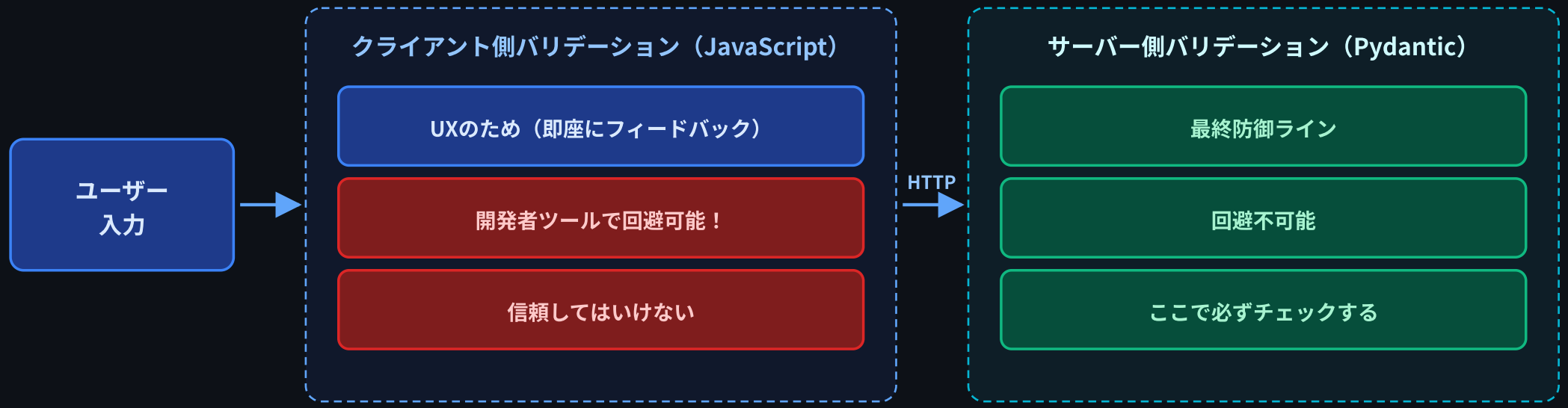
```
空文字 ""          →  タイトルのないTODOが作成される
超長文字列         →  データベースが圧迫される
不正なデータ型     →  サーバーエラーが発生する
```

バリデーション = 入力値のチェック

- **クライアント側**（JavaScript）: 即座にユーザーにフィードバック
- **サーバー側**（Python/Pydantic）: 最終的な防御ライン

両方で行う のがポイント！

クライアント側とサーバー側の両方で検証する理由



攻撃者はブラウザのJavaScriptを無効にしたり、
curlなどのツールでサーバーに直接リクエストを送ることができる。
クライアント側の検証だけでは不十分。

クライアント側バリデーション（JavaScript）

```javascript
function addTodo() {
  const input = document.getElementById('todo-input');
  const title = input.value.trim();

  // 空文字チェック
  if (title === '') {
    showError('TODOのタイトルを入力してください');
    return;
  }

  // 文字数チェック
  if (title.length > 100) {
    showError('タイトルは100文字以内で入力してください');
    return;
  }

  // 問題なければAPIを呼び出す
  fetch('/todos', {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify({ title: title })
  });
}
```

サーバー側バリデーション（Pydantic）

FastAPIはPydanticモデルで**自動バリデーション**ができる。

```
from pydantic import BaseModel, Field

class TodoCreate(BaseModel):
    title: str = Field(
        min_length=1,      # 1文字以上
        max_length=100,    # 100文字以下
        examples=["買い物に行く"]
    )

class TodoUpdate(BaseModel):
    done: bool

@app.post("/todos")
def create_todo(todo: TodoCreate):
    # Pydanticが自動的にバリデーションしてくれる
    # 不正な値の場合は 422 Unprocessable Entity が返される
    ...
```

バリデーションに失敗すると、FastAPIが自動的に
422エラーと分かりやすいエラーメッセージを返してくれる。

HTTPSの概念

HTTPとHTTPSの違い

```
HTTP（Hypertext Transfer Protocol）:
  ブラウザ ——— 平文（暗読可能）———→ サーバー
  ※ 通信内容が盗聴される可能性がある

HTTPS（HTTP Secure）:
  ブラウザ ——— 暗号化データ ———→ サーバー
  ※ 通信内容が暗号化され、盗聴されても読めない
```

- **HTTPS** = HTTP + **TLS/SSL**（暗号化の仕組み）
- 現在のWebでは **HTTPS が標準**（HTTPは非推奨）
- ブラウザのアドレスバーに鍵マークが表示される
- 本番環境では必ずHTTPSを使う

実習5（10分）：バリデーションを実装する

フロントエンド（exercise/static/app.js）

1. `addTodo` 関数に空文字チェックを追加
2. 文字数チェック（100文字以上は拒否）を追加
3. エラー時にメッセージを表示する `showError` 関数を作成

バックエンド（exercise/main.py）

4. `TodoCreate` モデルに `Field` バリデーションを追加

```
from pydantic import BaseModel, Field

class TodoCreate(BaseModel):
    title: str = Field(min_length=1, max_length=100)
```

5. Swagger UI（`/docs`）で空文字のPOSTリクエストを送り、エラーを確認

5. エラーハンドリング

エラーに適切に対応する

なぜエラーハンドリングが重要か

エラーハンドリングなしの場合

ユーザー: 存在しない
TODOを削除しようとする

→

サーバー:
500 Internal Server Error

→

ブラウザ: 何が起きたか
分からない
（無反応 or エラー画面）

→

ユーザー:
「アプリが壊れた？」
と不安になる

エラーハンドリングありの場合

ユーザー: 存在しない
TODOを削除しようとする

→

サーバー:
404 Not Found

→

ブラウザ:
「指定されたTODOは
見つかりませんでした」と表示

→

ユーザー:
「あ、既に削除されたのか」
と理解できる

HTTPステータスコードの使い分け

コード	意味	使い場面
200	OK	正常にレスポンスを返すとき
201	Created	リソースの新規作成成功時
400	Bad Request	リクエストの形式が不正なとき
404	Not Found	指定したリソースが存在しないとき
422	Unprocessable Entity	バリデーションエラー時
500	Internal Server Error	サーバー内部エラー

ポイント

- **200番台**: 成功
- **400番台**: クライアント側のエラー（入力ミスなど）
- **500番台**: サーバー側のエラー（バグなど）

サーバー側のエラーハンドリング（Python）

```python
from fastapi import FastAPI, HTTPException

@app.delete("/todos/{todo_id}")
def delete_todo(todo_id: int):
    conn = get_db_connection()
    cursor = conn.cursor()

    # まず対象のTODOが存在するか確認
    cursor.execute("SELECT * FROM todos WHERE id = ?", (todo_id,))
    todo = cursor.fetchone()

    if todo is None:
        conn.close()
        # 404エラーを返す
        raise HTTPException(
            status_code=404,
            detail="指定されたTODOは見つかりませんでした"
        )

    cursor.execute("DELETE FROM todos WHERE id = ?", (todo_id,))
    conn.commit()
    conn.close()
    return {"message": "削除しました"}
```

クライアント側のエラーハンドリング（JavaScript）

```javascript
async function deleteTodo(id) {
  try {
    const response = await fetch(`/todos/${id}`, {
      method: 'DELETE'
    });

    if (!response.ok) {
      // HTTPエラーの場合
      const error = await response.json();
      showError(error.detail || 'エラーが発生しました');
      return;
    }

    // 成功時: TODOリストを再描画
    await loadTodos();

  } catch (error) {
    // ネットワークエラー等の場合
    showError('通信エラーが発生しました。ネットワークを確認してください');
  }
}
```

try-catch / try-except の対比

JavaScript（クライアント側）

```javascript
try {
  const response = await fetch('/todos');
  const data = await response.json();
  renderTodos(data);
} catch (error) {
  showError('データの取得に失敗しました');
}
```

Python（サーバー側）

```python
try:
    conn = get_db_connection()
    cursor = conn.cursor()
    cursor.execute("SELECT * FROM todos")
    todos = cursor.fetchall()
except Exception as e:
    raise HTTPException(status_code=500, detail="データベースエラー")
finally:
    conn.close()
```

エラーメッセージの表示（JavaScript）

```javascript
function showError(message) {
  const errorDiv = document.getElementById('error-message');
  errorDiv.textContent = message;  // textContentを使う（XSS対策）
  errorDiv.style.display = 'block';
  setTimeout(() => {
    errorDiv.style.display = 'none';
  }, 3000);  // 3秒後に自動で非表示
}
```

エラーメッセージの表示（HTML / CSS）

```html
<!-- HTMLにエラーメッセージ表示エリアを追加 -->
<div id="error-message" class="error-message" style="display: none;"></div>
```

```css
.error-message {
  background-color: #fee2e2;
  color: #dc2626;
  padding: 12px 16px;
  border-radius: 8px;
  margin-bottom: 16px;
}
```

実習6（10分）: エラーハンドリングを実装する

バックエンド（exercise/main.py）

1. `delete_todo` で存在しないIDの場合 **404** を返すように修正
2. `update_todo` でも同様に 404 を返すように修正

フロントエンド（exercise/static/app.js）

3. `showError` 関数を実装
4. fetch の `response.ok` チェックを追加
5. try-catch でネットワークエラーもハンドリング

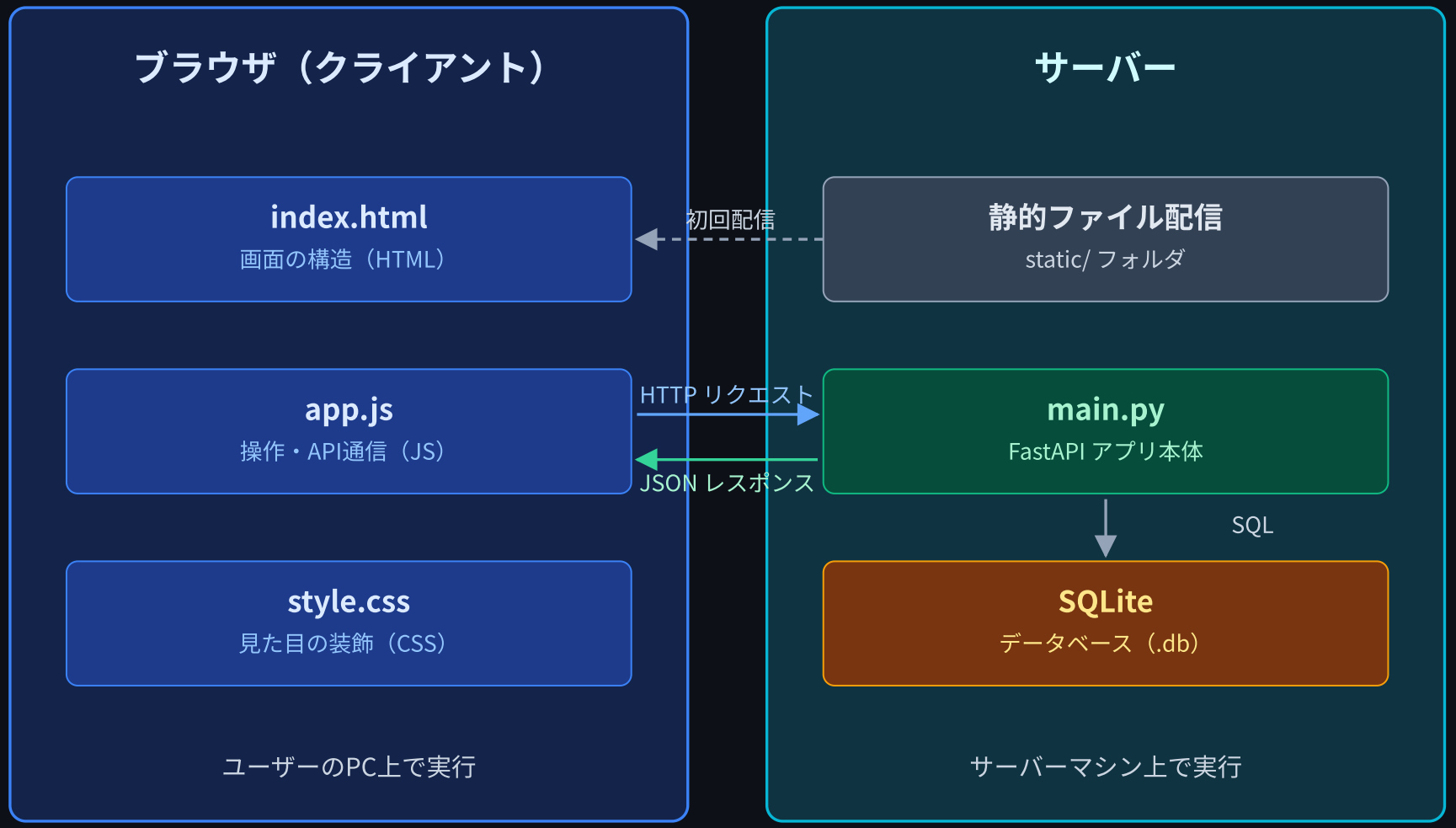
動作確認

6. 存在しないTODO（例: ID=9999）をSwagger UIで削除してみる → 404確認
7. ブラウザでエラーメッセージが表示されることを確認

6. 総まとめ

8回の振り返りと次のステップ

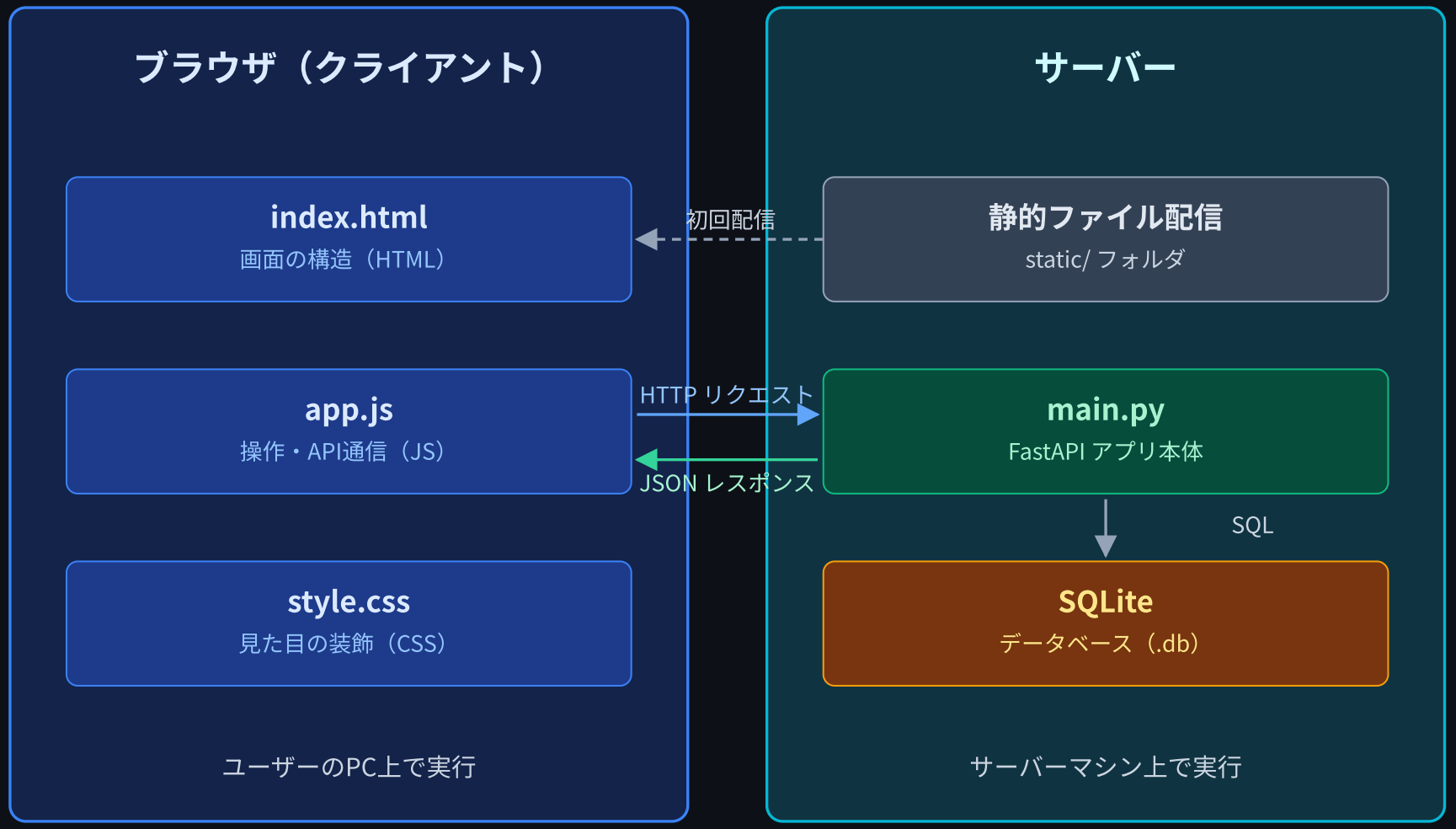
完成したTODOアプリのアーキテクチャ



各回で学んだ技術一覧

回	実行場所	学んだ技術
第1回	通信の仕組み	Git, GitHub, Codespaces, HTTP, 開発者ツール
第2回	ブラウザ	HTML（タグ, フォーム, 構造化）
第3回	ブラウザ	CSS（セレクタ, ボックスモデル, Flexbox, レスポンシブ）
第4回	ブラウザ	JavaScript（変数, 関数, DOM操作, イベント）
第5回	サーバー	Python（変数, 関数）, FastAPI, JSON
第6回	サーバー	SQLite, SQL（CRUD）, REST API設計
第7回	ブラウザ + サーバー	Fetch API, CORS, 静的ファイル配信
第8回	両方で防御	XSS, SQLインジェクション, バリデーション, エラーハンドリング

「どこで何が動いているか」の理解



次のステップ: フロントエンドを発展させる

この授業で基礎を身につけました。さらに学ぶなら:

技術	説明
TypeScript	JavaScriptに型安全性を追加する言語
React	Facebook製のUIライブラリ（コンポーネント指向）
Vue.js	学習しやすいフロントエンドフレームワーク
Tailwind CSS	ユーティリティファーストのCSSフレームワーク

次のステップ: バックエンドを発展させる

技術	説明
認証・認可	ログイン機能（JWT, OAuth）
PostgreSQL	より本格的なデータベース
Docker	アプリケーションのコンテナ化
クラウドデプロイ	AWS, GCP, Render, Vercel等での公開

心構え

- 「分からないこと」は正常。プロの開発者も毎日調べている
- 公式ドキュメントを読む習慣をつける
- シンプルな構造から徐々に複雑な機能を追加していく
- セキュリティは常に意識すること（特に公開する場合）

実習7（10分）: 最終 Git commit & push / 完成デモ

手順

1. すべての変更が保存されていることを確認
2. 完成したTODOアプリの動作確認
    - TODOの追加（正常な入力）
    - TODOの完了切替
    - TODOの削除
    - XSS入力が無効化されることの確認
    - 空文字入力でエラーメッセージが表示されること
    - 存在しないTODO操作でエラーメッセージが表示されること
3. 最終コミット

```
git add .
git commit -m "第8回: セキュリティ対策を追加してTODOアプリ完成"
git push
```

提出物

実習で穴埋め・実装を完了させたファイルをフォームから提出してください:

1. `main.py` のGitHubのURL
    - 例: `https://github.com/ユーザー名/リポジトリ名/blob/main/session08/exercise/main.py`
2. `app.js` のGitHubのURL
    - 例: `https://github.com/ユーザー名/リポジトリ名/blob/main/session08/exercise/static/app.js`

おめでとうございます！

8回の授業を通して、**Webアプリケーション**を完成させました。

```
あなたが作ったもの:
  HTML     → Webページの構造を作れるようになった
  CSS      → 見た目をデザインできるようになった
  JS       → 動的な処理を実装できるようになった
  Python   → サーバーサイドのプログラムを書けるようになった
  FastAPI  → REST APIを設計・実装できるようになった
  SQLite   → データベースでデータを管理できるようになった
  Security → 安全なアプリケーションの基礎を理解した
  Git      → バージョン管理とチーム開発の基礎を身につけた
```

これらはすべてのWeb開発の**基礎**です。

ここから先は、あなたの興味に合わせて自由に発展させてください。